DATA SCIENCE SOFTWARE ENGINEERING

Grounding Vibe Coding for Production ML & Deployment

Equipping Data Scientists to write AI-assisted, deployment-ready code.
Anchored in Ian Sommerville's *Software Engineering (10th Ed.)*.

PART 1

The Notebook-to-Production Gap

Why exploratory data science "vibe coding" stalls at the deployment team's doorstep.

Exploratory DS Code vs. Production ML

Exploratory Vibe Coding

Notebook Spaghetti: Out-of-order execution, global scope, and untracked cell dependencies.

Implicit Data Assumptions: Hardcoded paths, unvalidated nulls, and hidden data drift traps.

Monolithic Scripts: Mixing data fetching, feature engineering, modeling, and evaluation into one file.

Deployment-Ready Engineering

Modular Packages: Explicit Python modules, parameter configs, and versioned entry points.

Strict Data Contracts: Schema validation (Pandera / Pydantic) enforcing column types and bounds.

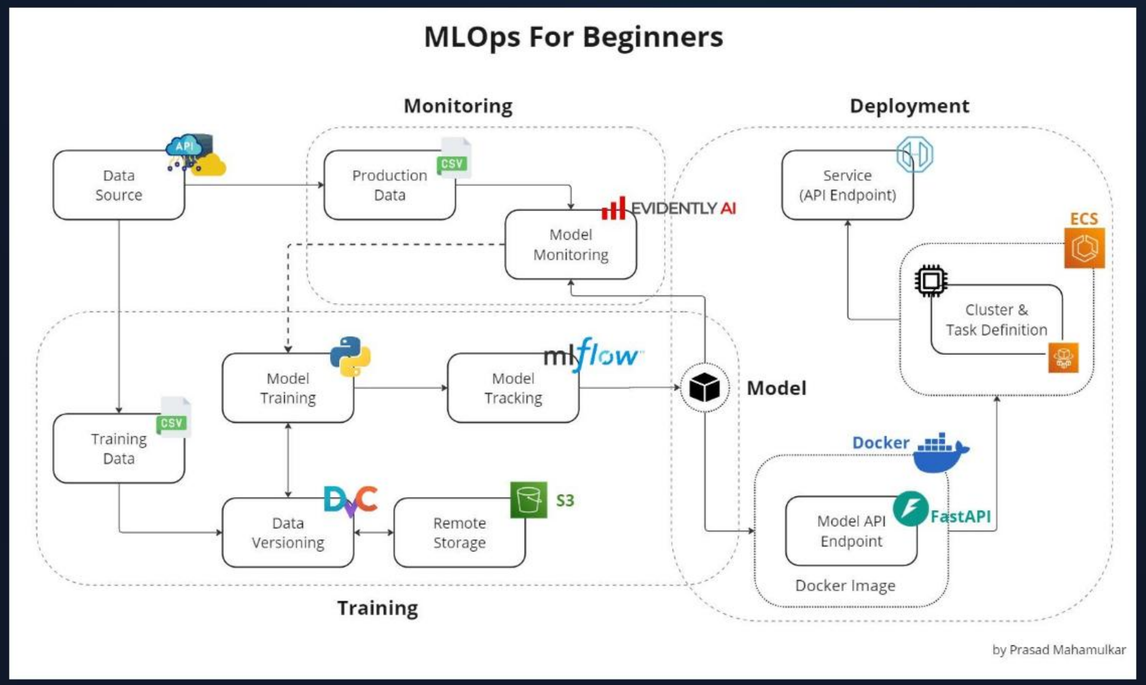
Clean Separation: Isolating ETL, training, inference, and API endpoints for MLOps integration.

Speaking the Deployment Team's Language

SOMMERVILLE CH. 1 & 4

System Contracts & Handoff

- **User Intent vs. System Contracts:** Data scientists optimize for metrics (F1/RMSE); MLOps teams optimize for SLAs, latency, and uptime.
- **Dependability by Design:** Error handling, logging, and retry logic must be engineered into prompts before model training code is written.
- **Boundary Conditions:** Define how the inference pipeline handles missing features, unexpected types, and model timeouts.



PART 2

Data Contracts & System Boundaries

Bridging high-level data manipulation with low-level production execution boundaries.

High-Level Modeling vs. Low-Level Execution

EXPLORATORY ABSTRACTION

High-Level DS Runtimes

Flexible In-Memory Processing: Pandas DataFrames, PyTorch tensors, interactive REPL environments.

Permissive Type Coercion: Silent column type conversions and auto-filled NaN values.

Generative Sweet Spot: LLMs excel at drafting Pandas/Scikit-Learn transformations rapidly.

PRODUCTION REALITIES

Low-Level Execution Limits

Strict Production Interfaces: C++ inference engines (Triton/TensorRT), gRPC, memory constraints.

Serialization & Latency: Binary protocols, GPU memory allocation, and vector alignment.

System Bottlenecks: Unvalidated DataFrame schemas cause runtime crashes in production microservices.

Data Contracts: Model APIs vs. Schemas

Dimension	Model API Contract (Pydantic / OpenAPI)	DataFrame Schema Contract (Pandera)
Scope	Inference HTTP/gRPC request payload	Batch ETL & Feature Engineering DataFrames
Validation	Field types, required keys, payload size limits	Column names, dtypes, value ranges, nullability
Enforcement	Validated at microservice API gateway before model execution	Validated during pipeline ingestion before training/scoring
Deployment Impact	Prevents 400/500 errors on serving endpoints	Prevents silent model corruption & feature drift in prod

PART 3

TDD & Modularization for ML Pipelines

Using Test-Driven Development and clean architecture to refactor prompt-generated code for production.

Test-Driven Development for Data Pipelines

SOMMERVILLE CH. 8

1. Write Tests First

Author unit tests for data cleaning, transformation, and feature extraction using synthetic mock data frames.

EXECUTION

2. Prompt Against Tests

Instruct the LLM to write vectorised PySpark/Polars transformation functions that pass test assertions.

REFACTORING

3. Refactor for Speed

Prompt AI to optimize memory usage and vectorization while automated pytest suites ensure zero logic regression.

The DS-to-Deployment Pipeline

1. Exploration

Rapid prototyping in notebooks with AI assist to prove statistical feasibility.

2. Package & Test

AI-driven refactoring into modular Python files with Pydantic/pytest validation.

3. CI/CD Validation

Automated tests, linting, data schema checks, and Docker containerization build.

4. MLOps Handoff

Deployment team deploys containerized inference endpoint with active monitoring.

The Deployment Golden Rule

0

Unvalidated Schemas in Prod

Bridging Data Science & MLOps

Never hand off notebook code to a deployment team without automated test coverage, explicit data contracts, and isolated environment configurations.

AI-generated code is technical debt until it passes deterministic assertions, type checking, and MLOps deployment review.

Deployment Best Practices Checklist for DS

- **Define Data Contracts First:** Use Pandera and Pydantic schemas to bound DataFrame inputs and API payloads.
- **Refactor Notebooks to Modules:** Package logic into standard Python packages with clean `src/` and `tests/` directories.
- **Enforce Pipeline TDD:** Author pytest assertions for feature transformations before asking AI to code them.
- **Lock Dependencies & Environments:** Provide reproducible Dockerfiles or lockfiles (`uv.lock`) for MLOps deployment.